Now, here's the worst part: if the data and mutex really are accessed by more than one thread, then as you add more cores and processors to the system, it becomes *more likely* that you will get high contention and one processor having to wait for another. If you're using multiple threads to process the same data more quickly, the threads are competing for the data and thus competing for the same mutex. The more of them there are, the more likely they'll try to acquire the mutex at the same time, or access the atomic variable at the same time, and so forth.

The effects of contention with mutexes are usually different from the effects of contention with atomic operations for the simple reason that the use of a mutex naturally serializes threads at the operating system level rather than at the processor level. If you have enough threads ready to run, the operating system can schedule another thread to run while one thread is waiting for the mutex, whereas a processor stall prevents any threads from running on that processor. However, it will still impact the performance of those threads that *are* competing for the mutex; they can only run one at a time, after all.

Back in chapter 3, you saw how a rarely updated data structure can be protected with a single-writer, multiple-reader mutex (see section 3.3.2). Cache ping-pong effects can nullify the benefits of such a mutex if the workload is unfavorable, because all threads accessing the data (even reader threads) still have to modify the mutex itself. As the number of processors accessing the data goes up, the contention on the mutex itself increases, and the cache line holding the mutex must be transferred between cores, thus potentially increasing the time taken to acquire and release locks to undesirable levels. There are techniques to ameliorate this problem, essentially by spreading out the mutex across multiple cache lines, but unless you implement your own such mutex, you are subject to whatever your system provides.

If this cache ping-pong is bad, how can you avoid it? As you'll see later in the chapter, the answer ties in nicely with general guidelines for improving the potential for concurrency: do what you can to reduce the potential for two threads competing for the same memory location.